

CSCE 637 COMPLEXITY THEORY · MODULE 05 · LECTURE NOTES

Beyond NP

co-NP, PSPACE, and the hierarchy.

MODULE OUTCOMES

- Define co-NP and explain the asymmetry.
- Define PSPACE and explain why games live there.
- Explain the polynomial hierarchy and what it measures.
- Place problems using quantifier alternation.
- Explain what collapses would mean.

1 co-NP

CO-NP IS 'SHORT CERTIFICATES FOR NO INSTANCES'

NP has short certificates for yes-instances. co-NP has short certificates for no-instances. A formula being satisfiable has an obvious witness — the assignment — and a formula being *unsatisfiable* has no obvious witness at all, since every assignment must fail.

So TAUTOLOGY and UNSAT are co-NP-complete, and whether $NP = co-NP$ is open — a genuinely separate question from P versus NP, though $P = NP$ would imply $NP = co-NP$ (since P is closed under complement, Module 02 §3).

And this is CSCE 627 Module 04 §2's recognisable/co-recognisable asymmetry with a resource bound attached — the same structural question, where in the unbounded setting it was settled (recognisable is not closed under complement, proved) and here it is open. Which is a good illustration of how adding resources makes questions harder rather than easier.

The practical trace is real: a SAT solver emitting an unsatisfiability proof is producing a co-NP certificate (CSCE 625 Module 08 §2) — and that is why those proofs can be enormous, sometimes terabytes. Whether short ones always exist is precisely the $NP = co-NP$ question, which makes proof complexity a direct attack on it.

Problem	Class	The certificate
SAT	NP-complete	A satisfying assignment, for a yes.
UNSAT, TAUTOLOGY	co-NP-complete	A refutation — which exists and may be exponentially long.

Problem	Class	The certificate
Primality	In P (Module 02 §2), and historically in $NP \cap co-NP$	A Pratt certificate for prime; a factor for composite. The $NP \cap co-NP$ membership was known decades before AKS, and was evidence it would turn out to be in P.
Factoring (decision version)	$NP \cap co-NP$	The factorisation certifies both directions. See the note.
Graph non-isomorphism	Not known to be in NP	No short certificate is known — but there is an interactive proof, which is Module 10 §1 and is how the subject got started.

A problem in $NP \cap co-NP$ is unlikely to be NP-complete, because if it were then every co-NP problem would reduce into NP and $NP = co-NP$ would follow. **Which is why factoring's membership in $NP \cap co-NP$ is evidence that it is NP-intermediate** (Module 03 §4) — **and it is a genuinely useful heuristic:** if your problem has short certificates in both directions, expect it to be in P or intermediate rather than complete, and look harder for a polynomial algorithm.

2 PSPACE

PSPACE IS POLYNOMIAL SPACE, AND IT CONTAINS THE GAMES

PSPACE is the class of problems decidable using polynomial space and unbounded time. **NP is contained in PSPACE**, because you can try every candidate certificate one at a time, reusing the same space — exponential time and polynomial space.

And TQBF — is a fully quantified Boolean formula true? — is PSPACE-complete. The difference from SAT is alternating quantifiers rather than a single block of existentials, and that alternation is what the extra power buys you.

Which is exactly why two-player games are PSPACE-complete: 'I have a move such that for all your replies I have a move such that...' is alternating quantification written out — and it is CSCE 625 Module 05 §1's minimax, which is now revealed as the canonical PSPACE computation.

Generalised geography, Go endgames, Reversi, and Hex are PSPACE-complete, while **chess and Go generalised to $n \times n$ boards are EXPTIME-complete** — harder still, **because those games can last exponentially many moves**, so the space to record the position history is not polynomial. **The distinction between the two classes of game is the game length**, which is a satisfying thing for a complexity class to be tracking.

```
L subset NL subset P subset NP subset PH
subset PSPACE subset EXPTIME subset EXPSPACE
```

EVERY INCLUSION SHOWN IS KNOWN.

NOT ONE of the inclusions from P up to PSPACE is known to be STRICT. $P = PSPACE$ is not ruled out, though essentially nobody believes it.

WHAT IS PROVED STRICT:

```
P != EXPTIME      (time hierarchy, Module 07)
NL != PSPACE      (space hierarchy)
L != PSPACE
```

SO THE HIERARCHY THEOREMS GIVE SEPARATIONS ONLY WHEN THE RESOURCE GAP IS LARGE -- exponential, not polynomial. And P vs NP is a question about machines with the SAME resource bound differing only in determinism, which is exactly what those theorems cannot address (Module 07 section 4).

THE PRACTICAL READING: "PSPACE-complete" means harder than NP-complete in all likelihood, and the solvers are correspondingly worse. A PSPACE-complete problem is where you should expect a solver to fail rather than to surprise you.

3 The polynomial hierarchy

THE POLYNOMIAL HIERARCHY IS CSCE 627'S ARITHMETIC HIERARCHY WITH POLYNOMIAL WITNESSES

Σ_1^P is NP: there exists a short witness. Π_1^P is co-NP: for all short witnesses.

Σ_2^P is 'there exists, then for all', and the alternation continues — exactly CSCE 627 Module 07 §2's hierarchy with the quantifiers restricted to polynomial-length strings and the inner predicate required to be polynomial-time rather than merely decidable. Same construction, resource-bounded.

Natural problems live at level 2: 'is this the minimum-size circuit computing this function?' is 'there exists a smaller circuit such that for all inputs it agrees' — negated, so it is Π_2^P . Which is why circuit minimisation is harder than SAT, and why nobody expects a SAT solver to do it.

And the hierarchy is believed strict and is not known to be. If any two adjacent levels coincide, the entire hierarchy collapses to that level — the proof is a straightforward induction — so $P = NP$ would collapse all of it to P, which is one reason the consequences in Module 03 §2 are so sweeping.

- **Circuit minimisation.** Π_2^P — see the callout.
- **Succinct set cover, and most 'is this the best X' questions.** Optimality is systematically one quantifier above existence, because 'best' means 'and nothing is better', which is a universal quantification over alternatives.
- **Two-player games with polynomially many moves. PSPACE-complete** — TQBF in disguise (§2).
- **Games of exponential length** — chess and Go generalised. EXPTIME-complete, and hence provably not in P.
- **And model checking against full temporal logic**, which sits in PSPACE — which is precisely why it is harder than SAT-based bounded model checking (CSCE 627 Module 08 §3), and why the bounded version is the one that scaled. 'Is this the best?' being one level harder than 'is there a good one?' is the most useful placing heuristic in this module, and it explains a great deal about which problems solvers handle.

4 What a collapse would mean

A COLLAPSE WOULD BE A STRUCTURAL SURPRISE, WHICH IS THE EVIDENCE AGAINST IT

If $\text{NP} = \text{co-NP}$, then unsatisfiability would have short certificates — and decades of proof-complexity work, which studies exactly the lengths of refutations in various proof systems, suggests strongly that it does not.

If the polynomial hierarchy collapses, then adding quantifier alternation buys no computational power — and CSCE 627 Module 07 showed that it does add power in the unbounded setting, where the question is settled. A resource-bounded version behaving entirely differently would be surprising.

So the arguments are structural: a collapse would make a natural-looking distinction vanish, and the field treats that as evidence against the collapse.

Which is weaker than proof, and is what is available. And the collapse results are used as evidence: 'if X were true, the polynomial hierarchy would collapse' is a standard and accepted argument that X is false — a conditional result deployed as a plausibility argument, which is how this field reasons in the absence of proof and is worth recognising as a style rather than mistaking for a theorem.

THE PRACTICAL CONTENT OF EVERYTHING ABOVE NP

NP-complete: solvers work, sometimes extremely well. Try one (Module 04 §4).

Above NP — Σ_2^P and beyond: expect solvers to struggle. QBF solvers exist and are substantially weaker than SAT solvers, which is the practical trace of a separation nobody has proved, and is reasonable circumstantial evidence for it.

PSPACE-complete: a solver will probably not help, and the engineering response is restriction, bounding, or approximation (CSCE 627 Module 12).

And the placing heuristic is quantifier alternation: write your problem's natural statement, count the quantifier alternations, and that is roughly where it sits — the same method as CSCE 627 Module 07 §2, and it takes a minute and is usually right.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Arora & Barak — chapters 4–5 (free draft)	co-NP, PSPACE, and the polynomial hierarchy — the reference for this module.
Sipser — chapter 8	PSPACE, TQBF, and the games , with the generalised geography reduction worked in full.
Hearn & Demaine — Games, Puzzles, and Computation	§2's games, systematically — which puzzles are NP-complete, PSPACE-complete, or EXPTIME-complete, and why. Library copy.

Resource	What it covers for this module
Beame & Pitassi — Propositional Proof Complexity (free survey)	§4's first argument — why short refutations are not expected, which is the main line of attack on NP versus co-NP.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Place five problems in NP, co-NP, or both**, naming the certificate in each direction.
- 2 **Explain why a problem in $NP \cap co-NP$ is unlikely to be NP-complete.**
- 3 **Get an unsatisfiability proof from a SAT solver** and report its size relative to the instance.
- 4 **Show $NP \subseteq PSPACE$** by the space-reuse argument.
- 5 **Reduce a two-player game to TQBF**, or sketch the correspondence.
- 6 **Write the landscape from memory** and mark which separations are known.
- 7 **Place three problems in the polynomial hierarchy** by counting quantifier alternations.
- 8 **Take an optimisation problem and its decision version** and show the optimality question is one level higher.
- 9 **Run a QBF solver and a SAT solver** on comparable instances and compare.
- 10 **Find a published argument of the form 'X would collapse the hierarchy'** and assess it.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define co-NP and explain the asymmetry.
- 2 What is the practical trace of co-NP in a SAT solver?
- 3 Why is a problem in $NP \cap co-NP$ unlikely to be complete?
- 4 Define PSPACE and explain why games live there.
- 5 Why are chess and Go EXPTIME-complete rather than PSPACE-complete?
- 6 Write the class landscape and say which separations are known.
- 7 Why do the hierarchy theorems not settle P versus NP?
- 8 Define the polynomial hierarchy and relate it to CSCE 627's.
- 9 What happens if two adjacent levels coincide?
- 10 Give the placing heuristic and the practical reading of each class.